Практическая работа: Проектируем логику службы каршеринга электрокаров

Требование 1

```
USE gol;
GO

-- Триггер: автоматическая запись в RentalStatusLog при смене статуса аренды
CREATE OR ALTER TRIGGER trg_RentalStatusLog
ON dbo.Rentals
AFTER UPDATE
AS
BEGIN
   SET NOCOUNT ON;
   -- Срабатываем только при изменении поля Status
   IF NOT UPDATE(Status) RETURN;

   INSERT INTO dbo.RentalStatusLog(RentalId, OldStatus, NewStatus, ChangedAt,
ChangedBy)
   SELECT i.RentalId, d.Status, i.Status, GETDATE(), SYSTEM_USER
   FROM inserted i
   JOIN deleted d ON i.RentalId = d.RentalId
   WHERE i.Status <> d.Status;
END
GO

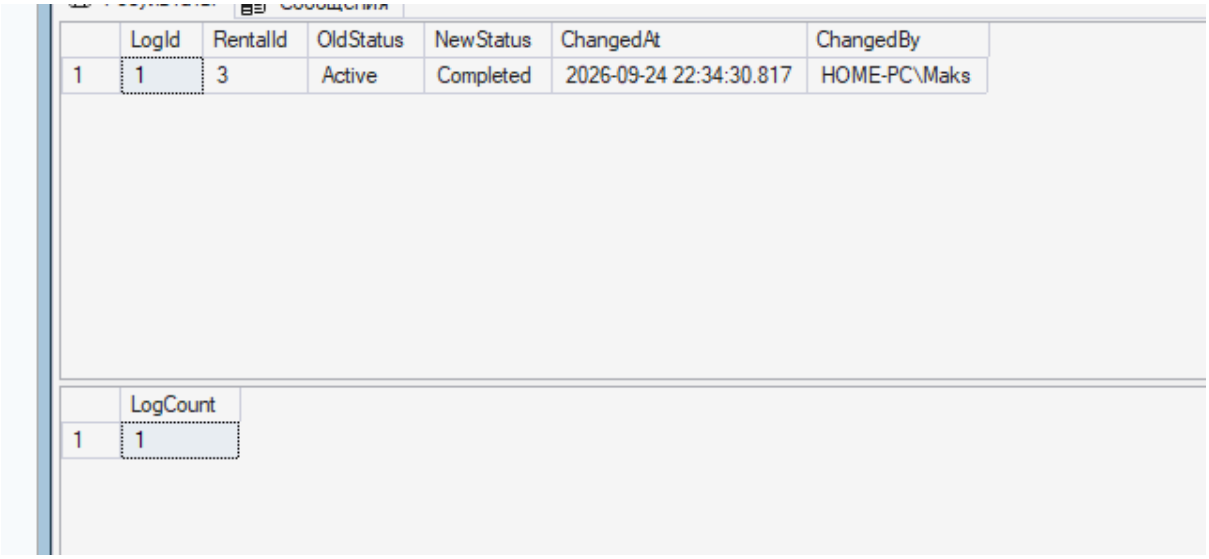
-- Тест 1.1: Успешная смена статуса аренды (должна появиться запись в журнале)
PRINT '--- Тест 1.1: Смена статуса аренды ---';
UPDATE dbo.Rentals SET Status = 'Completed' WHERE RentalId = 3;

PRINT 'Журнал статусов:';
SELECT * FROM dbo.RentalStatusLog ORDER BY LogId DESC;
GO

-- Тест 1.2: Изменение другого поля (триггер НЕ должен срабатывать)
PRINT '--- Тест 1.2: Изменение ExtraCost (статус не меняется) ---';
UPDATE dbo.Rentals SET ExtraCost = 100 WHERE RentalId = 4;

PRINT 'Количество записей в журнале (должно быть 1):';
SELECT COUNT(*) AS LogCount FROM dbo.RentalStatusLog;
GO
```

Скрин:



	LogId	RentalId	OldStatus	NewStatus	ChangedAt	ChangedBy
1	1	3	Active	Completed	2026-09-24 22:34:30.817	HOME-PC\Maks

	LogCount
1	1

Требование 2

```
USE gol;
GO

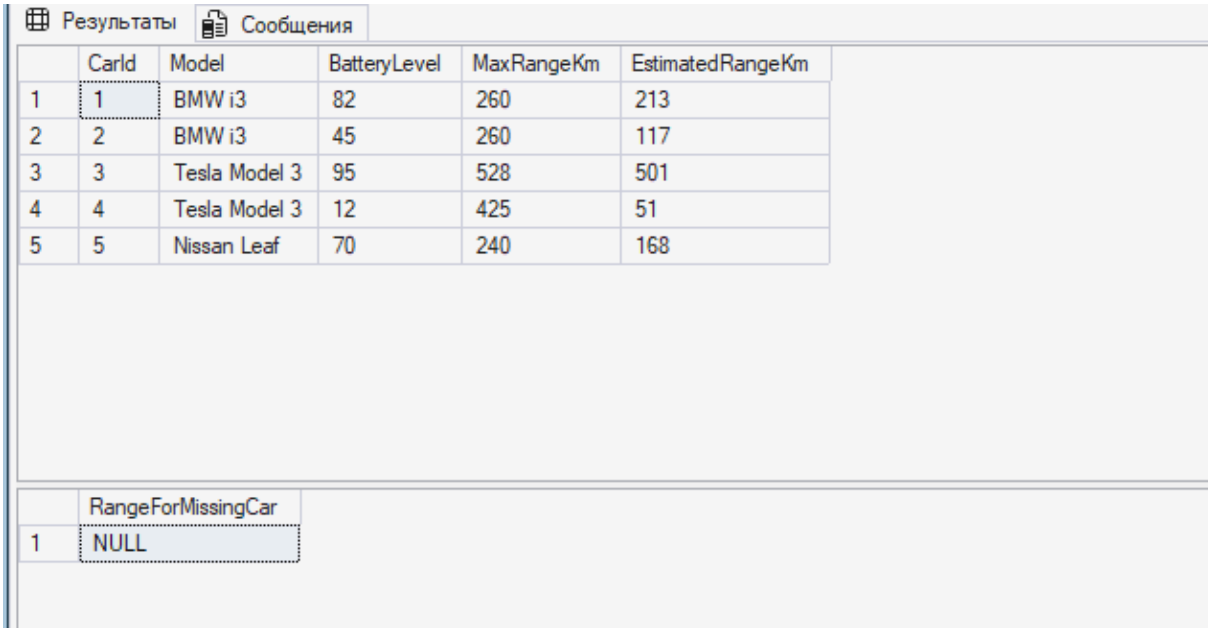
-- Функция: расчёт запаса хода = MaxRangeKm * (BatteryLevel / 100)
CREATE OR ALTER FUNCTION dbo.GetEstimatedRange
(
    @CarId INT
)
RETURNS INT
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @MaxRange INT, @BatteryLevel INT;
    SELECT @MaxRange = MaxRangeKm, @BatteryLevel = BatteryLevel
    FROM dbo.Cars WHERE CarId = @CarId;
    IF @MaxRange IS NULL OR @BatteryLevel IS NULL RETURN NULL;
    RETURN @MaxRange * @BatteryLevel / 100;
END
GO

-- Тест 2.1: Запас хода для всех авто
PRINT '--- Тест 2.1: Запас хода всех авто ---';
SELECT CarId, Model, BatteryLevel, MaxRangeKm,
       dbo.GetEstimatedRange(CarId) AS EstimatedRangeKm
FROM dbo.Cars;
GO

-- Тест 2.2: Несуществующее авто (должно вернуть NULL)
PRINT '--- Тест 2.2: Несуществующее авто ---';
SELECT dbo.GetEstimatedRange(999) AS RangeForMissingCar;
```

GO

Скрин:


Результаты Сообщения

	CarId	Model	BatteryLevel	MaxRangeKm	EstimatedRangeKm
1	1	BMW i3	82	260	213
2	2	BMW i3	45	260	117
3	3	Tesla Model 3	95	528	501
4	4	Tesla Model 3	12	425	51
5	5	Nissan Leaf	70	240	168

	RangeForMissingCar
1	NULL

Требование 3

```
USE gol;
GO

-- Процедура: оформление аренды с проверками
CREATE OR ALTER PROCEDURE dbo.StartRental
    @CustomerId INT,
    @CarId INT,
    @DriverId INT,
    @StartOdometer DECIMAL(10,2),
    @BaseCost DECIMAL(10,2),
    @NewRentalId INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        -- Проверка: авто в статусе Available
        IF NOT EXISTS (SELECT 1 FROM dbo.Cars WHERE CarId = @CarId AND Status =
'Available')
            THROW 50001, N'Автомобиль недоступен для аренды', 1;

        -- Проверка: лицензия водителя не истекла
```

```
        IF EXISTS (SELECT 1 FROM dbo.Drivers WHERE DriverId = @DriverId AND
LicenseExpiry < GETDATE())
            THROW 50002, N'Лицензия водителя истекла', 1;

        -- Проверка: заряда хватит минимум на 30% от MaxRangeKm
        IF NOT EXISTS (
            SELECT 1 FROM dbo.Cars
            WHERE CarId = @CarId AND (BatteryLevel / 100.0) * MaxRangeKm >=
MaxRangeKm * 0.3
        )
            THROW 50003, N'Недостаточный заряд для аренды (нужно минимум 30%)', 1;

        -- Создаём аренду
        INSERT INTO dbo.Rentals(CustomerId, CarId, DriverId, StartAt, StartOdometer,
BaseCost, Status, CreatedAt)
        VALUES (@CustomerId, @CarId, @DriverId, GETDATE(), @StartOdometer,
@BaseCost, 'Active', GETDATE());

        SET @NewRentalId = SCOPE_IDENTITY();

        -- Меняем статус авто на Rented
        UPDATE dbo.Cars SET Status = 'Rented' WHERE CarId = @CarId;

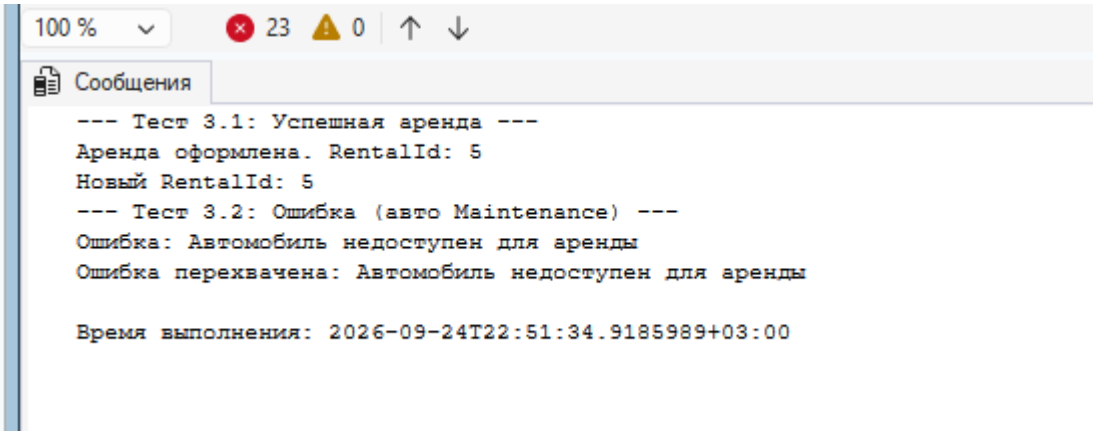
        COMMIT TRANSACTION;
        PRINT N'Аренда оформлена. RentalId: ' + CAST(@NewRentalId AS
NVARCHAR(10));
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        PRINT N'Ошибка: ' + ERROR_MESSAGE();
        THROW;
    END CATCH
END
GO

-- Тест 3.1: Успешная аренда (авто #1 — BMW i3, Available)
PRINT '--- Тест 3.1: Успешная аренда ---';
DECLARE @NewRental INT;
EXEC dbo.StartRental
    @CustomerId = 1, @CarId = 1, @DriverId = 1,
    @StartOdometer = 15300.0, @BaseCost = 1500.00,
    @NewRentalId = @NewRental OUTPUT;
PRINT 'Новый RentalId: ' + CAST(@NewRental AS NVARCHAR(10));
GO

-- Тест 3.2: Ошибка — авто в статусе Maintenance (Nissan Leaf #5)
PRINT '--- Тест 3.2: Ошибка (авто Maintenance) ---';
BEGIN TRY
```

```
    DECLARE @Dummy INT;
    EXEC dbo.StartRental
        @CustomerId = 1, @CarId = 5, @DriverId = 1,
        @StartOdometer = 58100.0, @BaseCost = 1000.00,
        @NewRentalId = @Dummy OUTPUT;
END TRY
BEGIN CATCH
    PRINT 'Ошибка перехвачена: ' + ERROR_MESSAGE();
END CATCH;
GO
```

Скрин:



Требование 4

```
USE gol;
GO

-- Сначала проверим статусы всех аренд
PRINT 'Текущие статусы аренд:';
SELECT RentalId, CarId, DriverId, Status FROM dbo.Rentals;
GO

-- Процедура: завершение аренды с расчётами
CREATE OR ALTER PROCEDURE dbo.CompleteRental
    @RentalId INT,
    @EndOdometer DECIMAL(10,2),
    @ExtraCost DECIMAL(10,2) = 0
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
```

```
    DECLARE @CarId INT, @DriverId INT, @StartOdometer DECIMAL(10,2), @BaseCost
DECIMAL(10,2);
    DECLARE @Distance DECIMAL(10,2), @TotalCost DECIMAL(10,2);

    -- Получаем данные активной аренды
    SELECT @CarId = CarId, @DriverId = DriverId,
        @StartOdometer = StartOdometer, @BaseCost = BaseCost
    FROM dbo.Rentals
    WHERE RentalId = @RentalId AND Status = 'Active';

    -- Проверка: аренда существует и активна
    IF @CarId IS NULL
      THROW 50010, N'Аренда не найдена или уже завершена', 1;

    -- Расчёты
    SET @Distance = @EndOdometer - @StartOdometer;
    SET @TotalCost = @BaseCost + @ExtraCost;

    -- Обновляем аренду
    UPDATE dbo.Rentals
    SET EndAt = GETDATE(), EndOdometer = @EndOdometer,
      DistanceCovered = @Distance, TotalCost = @TotalCost, Status = 'Completed'
    WHERE RentalId = @RentalId;

    -- Обновляем пробег и статус авто
    UPDATE dbo.Cars
    SET CurrentMileage = CurrentMileage + @Distance, Status = 'Available'
    WHERE CarId = @CarId;

    -- Обновляем статистику водителя
    UPDATE dbo.Drivers
    SET TotalRents = TotalRents + 1, TotalDistance = TotalDistance + @Distance
    WHERE DriverId = @DriverId;

    COMMIT TRANSACTION;
    PRINT N'Аренда ' + CAST(@RentalId AS NVARCHAR(10)) + N' завершена. Пробег: '
+
        CAST(@Distance AS NVARCHAR(20)) + N' км, Стоимость: ' + CAST(@TotalCost
AS NVARCHAR(20));
  END TRY
  BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    PRINT N'Ошибка: ' + ERROR_MESSAGE();
    THROW;
  END CATCH
END
GO
```

```
-- Тест 4.1: Успешное завершение аренды #4 (она ещё Active)
PRINT '--- Тест 4.1: Завершение аренды #4 ---';
EXEC dbo.CompleteRental @RentalId = 4, @EndOdometer = 15350.0, @ExtraCost =
150.00;

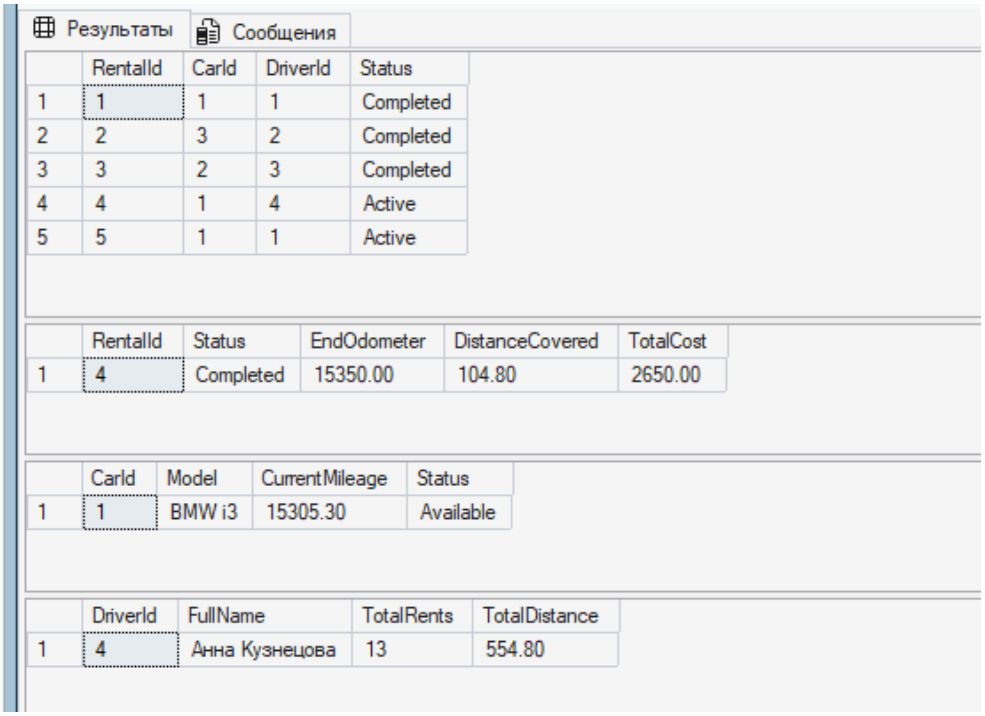
-- Проверяем результат
PRINT 'Статус аренды #4:';
SELECT RentalId, Status, EndOdometer, DistanceCovered, TotalCost
FROM dbo.Rentals WHERE RentalId = 4;

PRINT 'Пробег авто #1:';
SELECT CarId, Model, CurrentMileage, Status FROM dbo.Cars WHERE CarId = 1;

PRINT 'Статистика водителя #4:';
SELECT DriverId, FullName, TotalRents, TotalDistance FROM dbo.Drivers WHERE DriverId
= 4;
GO

-- Тест 4.2: Повторное завершение той же аренды (ошибка)
PRINT '--- Тест 4.2: Повторное завершение ---';
BEGIN TRY
    EXEC dbo.CompleteRental @RentalId = 4, @EndOdometer = 15400.0;
END TRY
BEGIN CATCH
    PRINT 'Ошибка перехвачена: ' + ERROR_MESSAGE();
END CATCH;
GO
```

Скрин:



Результаты | Сообщения

	RentalId	CarId	DriverId	Status
1	1	1	1	Completed
2	2	3	2	Completed
3	3	2	3	Completed
4	4	1	4	Active
5	5	1	1	Active

	RentalId	Status	EndOdometer	DistanceCovered	TotalCost
1	4	Completed	15350.00	104.80	2650.00

	CarId	Model	CurrentMileage	Status
1	1	BMW i3	15305.30	Available

	DriverId	FullName	TotalRents	TotalDistance
1	4	Анна Кузнецова	13	554.80

Требование 5

```
USE gol;

GO


-- Триггер: автоматический перевод авто в статус Charging при заряде < 20%

CREATE OR ALTER TRIGGER trg_AutoCharging

ON dbo.Cars

AFTER UPDATE

AS

BEGIN

    SET NOCOUNT ON;


    -- Срабатываем только при изменении BatteryLevel

    IF NOT UPDATE(BatteryLevel) RETURN;


    -- Если заряд упал ниже 20% — переводим в Charging

    UPDATE c

    SET c.Status = 'Charging'

    FROM dbo.Cars c

    JOIN inserted i ON c.CarId = i.CarId

    WHERE i.BatteryLevel < 20 AND c.Status <> 'Charging';


    IF @@ROWCOUNT > 0
```

```
        PRINT N'Автомобиль переведён в статус Charging (заряд < 20%)';
END
GO

-- Тест 5.1: Снижение заряда ниже 20% (должен сработать триггер)
PRINT '--- Тест 5.1: Авто-перевод в зарядку ---';
PRINT 'До изменения:';
SELECT CarId, Model, BatteryLevel, Status FROM dbo.Cars WHERE CarId = 2;

UPDATE dbo.Cars SET BatteryLevel = 15 WHERE CarId = 2;

PRINT 'После изменения:';
SELECT CarId, Model, BatteryLevel, Status FROM dbo.Cars WHERE CarId = 2;
GO

-- Тест 5.2: Снижение заряда, но выше 20% (триггер НЕ должен менять статус)
PRINT '--- Тест 5.2: Заряд выше 20% ---';
PRINT 'До изменения:';
SELECT CarId, Model, BatteryLevel, Status FROM dbo.Cars WHERE CarId = 1;

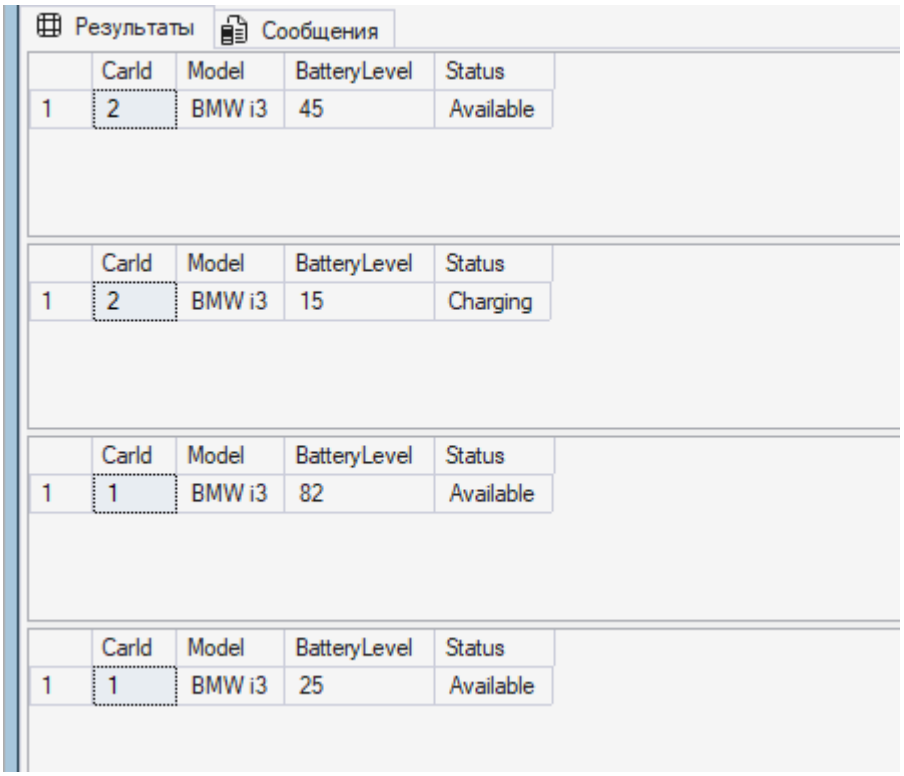
UPDATE dbo.Cars SET BatteryLevel = 25 WHERE CarId = 1;

PRINT 'После изменения:';
```

```
SELECT CarId, Model, BatteryLevel, Status FROM dbo.Cars WHERE CarId = 1;

GO
```

Скрин:



Результаты | Сообщения

	CarId	Model	BatteryLevel	Status
1	2	BMW i3	45	Available

	CarId	Model	BatteryLevel	Status
1	2	BMW i3	15	Charging

	CarId	Model	BatteryLevel	Status
1	1	BMW i3	82	Available

	CarId	Model	BatteryLevel	Status
1	1	BMW i3	25	Available

Требование 6

```
USE gol;
GO

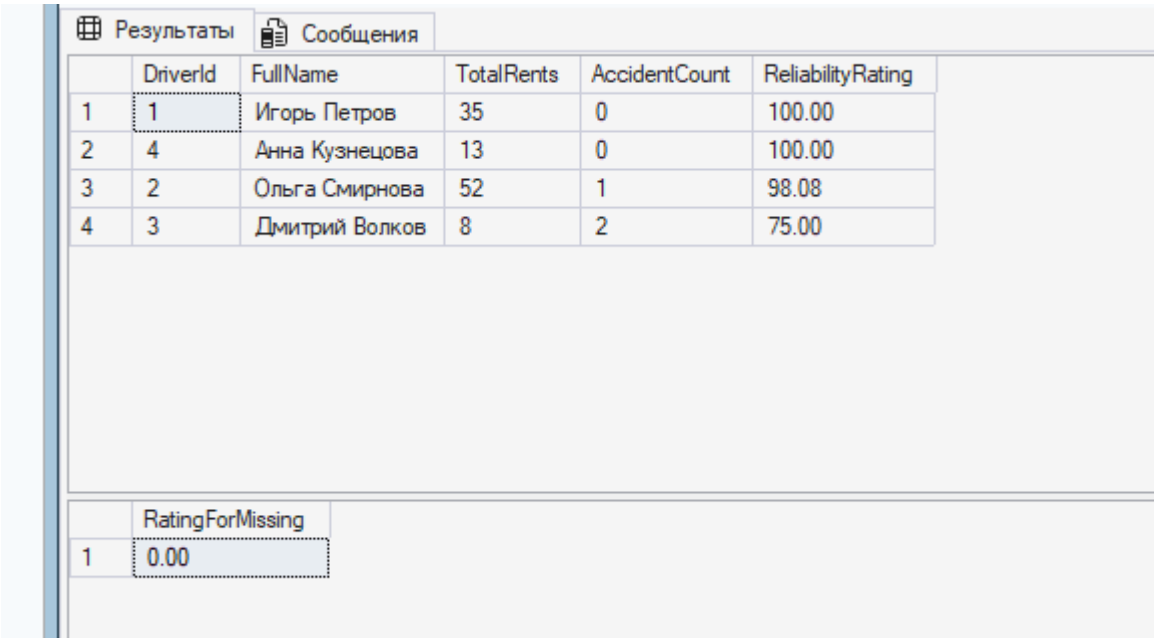
-- Функция: рейтинг надёжности = ((TotalRents - AccidentCount) / TotalRents) * 100
CREATE OR ALTER FUNCTION dbo.GetDriverReliability
(
    @DriverId INT
)
RETURNS DECIMAL(5,2)
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @TotalRents INT, @Accidents INT, @Rating DECIMAL(5,2);
    SELECT @TotalRents = TotalRents, @Accidents = AccidentCount
    FROM dbo.Drivers WHERE DriverId = @DriverId;
    IF @TotalRents IS NULL OR @TotalRents = 0 RETURN 0;
```

```
    SET @Rating = CAST((@TotalRents - @Accidents) AS DECIMAL(5,2)) / @TotalRents *
100;
    RETURN @Rating;
END
GO

-- Тест 6.1: Рейтинг всех водителей
PRINT '--- Тест 6.1: Рейтинг водителей ---';
SELECT DriverId, FullName, TotalRents, AccidentCount,
       dbo.GetDriverReliability(DriverId) AS ReliabilityRating
FROM dbo.Drivers
ORDER BY ReliabilityRating DESC;
GO

-- Тест 6.2: Несуществующий водитель (должен вернуть 0)
PRINT '--- Тест 6.2: Несуществующий водитель ---';
SELECT dbo.GetDriverReliability(999) AS RatingForMissing;
GO
```

Скрин:



	DriverId	FullName	TotalRents	AccidentCount	ReliabilityRating
1	1	Игорь Петров	35	0	100.00
2	4	Анна Кузнецова	13	0	100.00
3	2	Ольга Смирнова	52	1	98.08
4	3	Дмитрий Волков	8	2	75.00

	RatingForMissing
1	0.00

Требование 7

```
USE gol;
GO

-- Сначала посмотрим текущие данные зарядки
PRINT 'Текущие записи ChargingLogs:';
SELECT * FROM dbo.ChargingLogs;
GO
```

```
-- Обновляем LogId=2: добавляем ChargeTo = 95 (зарядка завершена до 95%)
UPDATE dbo.ChargingLogs
SET ChargeTo = 95
WHERE LogId = 2 AND ChargeTo IS NULL;
PRINT 'LogId=2 обновлён: ChargeTo = 95';
GO

-- Теперь выполняем тест 7.1
PRINT '--- Тест 7.1: Завершение зарядки ---';
PRINT 'До зарядки:';
SELECT CarId, Model, BatteryLevel, Status FROM dbo.Cars WHERE CarId = 4;

EXEC dbo.CompleteCharging @LogId = 2;

PRINT 'После зарядки:';
SELECT CarId, Model, BatteryLevel, Status FROM dbo.Cars WHERE CarId = 4;

PRINT 'Лог зарядки:';
SELECT * FROM dbo.ChargingLogs WHERE LogId = 2;
GO

-- Тест 7.2: Ошибка — ChargeTo < ChargeFrom
PRINT '--- Тест 7.2: Ошибка (ChargeTo < ChargeFrom) ---';
INSERT INTO dbo.ChargingLogs(CarId, ChargingStart, ChargeFrom, ChargeTo, Location)
VALUES (1, GETDATE(), 80, 50, N'Тест');

BEGIN TRY
   EXEC dbo.CompleteCharging @LogId = SCOPE_IDENTITY();
END TRY
BEGIN CATCH
   PRINT 'Ошибка перехвачена: ' + ERROR_MESSAGE();
END CATCH;
GO
```

Скрин:

```
Текущие записи ChargingLogs:

(затронуто записей: 2)

(затронута 1 запись)
LogId=2 обновлён: ChargeTo = 95
--- Тест 7.1: Завершение зарядки ---
До зарядки:

(затронута 1 запись)
Зарядка завершена. Авто 4, заряд восстановлен до 95%
После зарядки:

(затронута 1 запись)
Лог зарядки:
```

```
--- Тест 7.2: Ошибка (ChargeTo < ChargeFrom) ---

(затронута 1 запись)
Создана запись зарядки с LogId = 3
Ошибка: ChargeTo не может быть меньше ChargeFrom
Ошибка перехвачена: ChargeTo не может быть меньше ChargeFrom
```

Требование 8

```
USE gol;
GO

-- Триггер: блокировка назначения авто в Maintenance/Accident/Charging
-- Срабатывает на INSERT и UPDATE таблицы Rentals
CREATE OR ALTER TRIGGER trg_PreventFaultyCarRental
ON dbo.Rentals
INSTEAD OF INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Проверка: нет ли среди назначаемых авто запрещённых статусов
    IF EXISTS (
        SELECT 1 FROM inserted i
        JOIN dbo.Cars c ON i.CarId = c.CarId
        WHERE c.Status IN ('Maintenance', 'Accident', 'Charging')
    )
    BEGIN
        RAISERROR(N'Нельзя назначить автомобиль в статусе Maintenance, Accident или
Charging на аренду', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END
```

```
    -- Если это INSERT — выполняем вставку
    IF NOT EXISTS (SELECT 1 FROM deleted)
    BEGIN
        INSERT INTO dbo.Rentals(CustomerId, CarId, DriverId, StartAt, EndAt,
            StartOdometer, EndOdometer, DistanceCovered, BaseCost, ExtraCost, TotalCost,
Status, CreatedAt)
        SELECT CustomerId, CarId, DriverId, StartAt, EndAt,
            StartOdometer, EndOdometer, DistanceCovered, BaseCost, ExtraCost, TotalCost,
Status, CreatedAt
        FROM inserted;
    END
    -- Если UPDATE — выполняем обновление
    ELSE
    BEGIN
        UPDATE r
        SET r.CustomerId = i.CustomerId,
            r.DriverId = i.DriverId,
            r.StartAt = i.StartAt,
            r.EndAt = i.EndAt,
            r.StartOdometer = i.StartOdometer,
            r.EndOdometer = i.EndOdometer,
            r.DistanceCovered = i.DistanceCovered,
            r.BaseCost = i.BaseCost,
            r.ExtraCost = i.ExtraCost,
            r.TotalCost = i.TotalCost,
            r.Status = i.Status
        FROM dbo.Rentals r
        JOIN inserted i ON r.RentalId = i.RentalId;
    END
END
GO

-- Тест 8.1: Успешный UPDATE (меняем ExtraCost на аренду с Available авто)
PRINT '--- Тест 8.1: Успешный UPDATE ---';
UPDATE dbo.Rentals SET ExtraCost = 300 WHERE RentalId = 1;
PRINT 'UPDATE прошёл успешно';

-- Проверяем результат
SELECT RentalId, ExtraCost FROM dbo.Rentals WHERE RentalId = 1;
GO

-- Тест 8.2: ОШИБКА — назначение авто #5 (Maintenance) на аренду #4
PRINT '--- Тест 8.2: Блокировка неисправного авто ---';
PRINT 'Попытка назначить авто #5 (Maintenance) на аренду #4:';

UPDATE dbo.Rentals SET CarId = 5 WHERE RentalId = 4;
GO
```

Скрин:

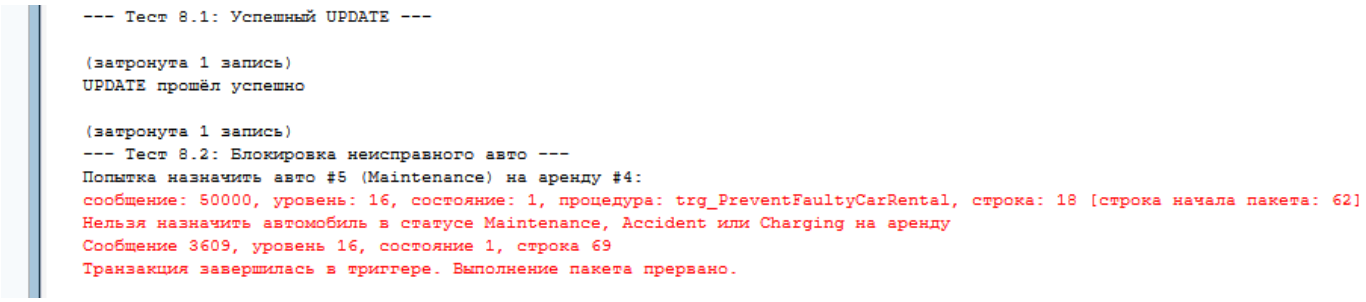


Таблица обоснований

№	Выбранный объект	Почему этот, а не другой?	Почему альтернативы не подходят?
1	AFTER UPDATE триггер на Rentals	Логика должна срабатывать автоматически при любом изменении статуса, даже при прямом UPDATE	Процедура не сработает при прямом UPDATE; функция не может вставлять данные
2	Скалярная функция	Значение должно быть доступно в любом SELECT и отчетах	Триггер не возвращает значения; процедура не используется в SELECT
3	Хранимая процедура	Многошаговый процесс с валидацией, транзакцией и обработкой ошибок	Триггер не подходит для инициации операции; функция не может изменять данные

4	Хранимая процедура	Атомарный процесс с множеством обновлений и обработкой ошибок	Триггер не инициирует завершение; функция не выполняет DML
5	AFTER UPDATE триггер на Cars	Должно срабатывать автоматически при снижении заряда	Процедура требует ручного вызова; функция не меняет данные
6	Скалярная функция	Рейтинг нужен в SELECT и отчётах	Триггер/процедура не возвращают значения для SELECT
7	Хранимая процедура	Валидация + обновление + транзакция	Триггер не инициирует процесс; функция не выполняет DML
8	INSTEAD OF INSERT/UPDATE триггер на Rentals	Должен блокировать даже прямой UPDATE со стороны приложения	Процедура не защитит от прямого UPDATE; функция не блокирует изменения